



Edwin Bermúdez

CONSULTOR DEVOPS & SRE

CONTROLES EMPRESARIALES

+10 Años - Experiencia IT

+6 Años - DevOps & SRE

PERFIL PROFESIONAL

Ingeniero electrónico con más de 10 años de experiencia en TI. Especialista en diseñar y operar plataformas Cloud-Native resilientes, automatizadas y escalables. Trayectoria líder en Kubernetes, IaC, CI/CD y entornos híbridos multicloud, con la distinción de Kubestronaut por el dominio del ecosistema



LINKEDIN

/edwin-bermudez-cuervo



GITHUB

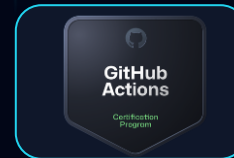
edwinbermudez91



EMAIL

ebermudez@coem.co

CERTIFICACIONES ELITE



SRE

IaC

CI/CD

Multicloud

GitOps

Observability



Fundamentos de Docker



¿Qué es un contenedor? Diferencias con máquinas virtuales.

Arquitectura de Docker (cliente, daemon, registry).

Imágenes y contenedores.

Dockerfile y construcción de imágenes.

Comandos básicos.

Redes y volúmenes.

Multi-stage builds.

Laboratorio guiado.

¿Qué es un contenedor?



Los contenedores empaquetan una aplicación con todas sus dependencias (bibliotecas, configuración) en una unidad ligera.



Comparten el kernel del sistema operativo anfitrión.

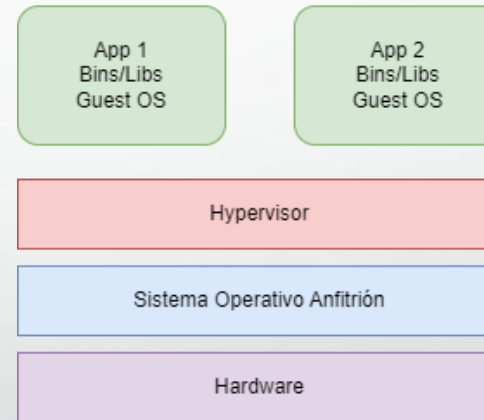


A diferencia de las máquinas virtuales, no incluyen un sistema operativo completo.

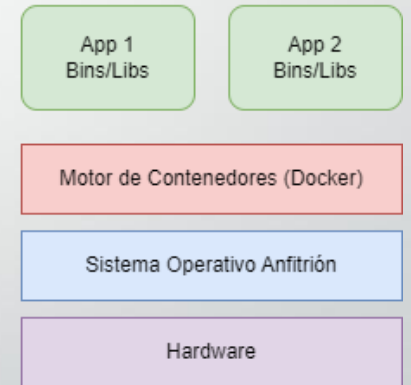


Arranque en milisegundos, menor consumo de recursos.

Máquinas Virtuales

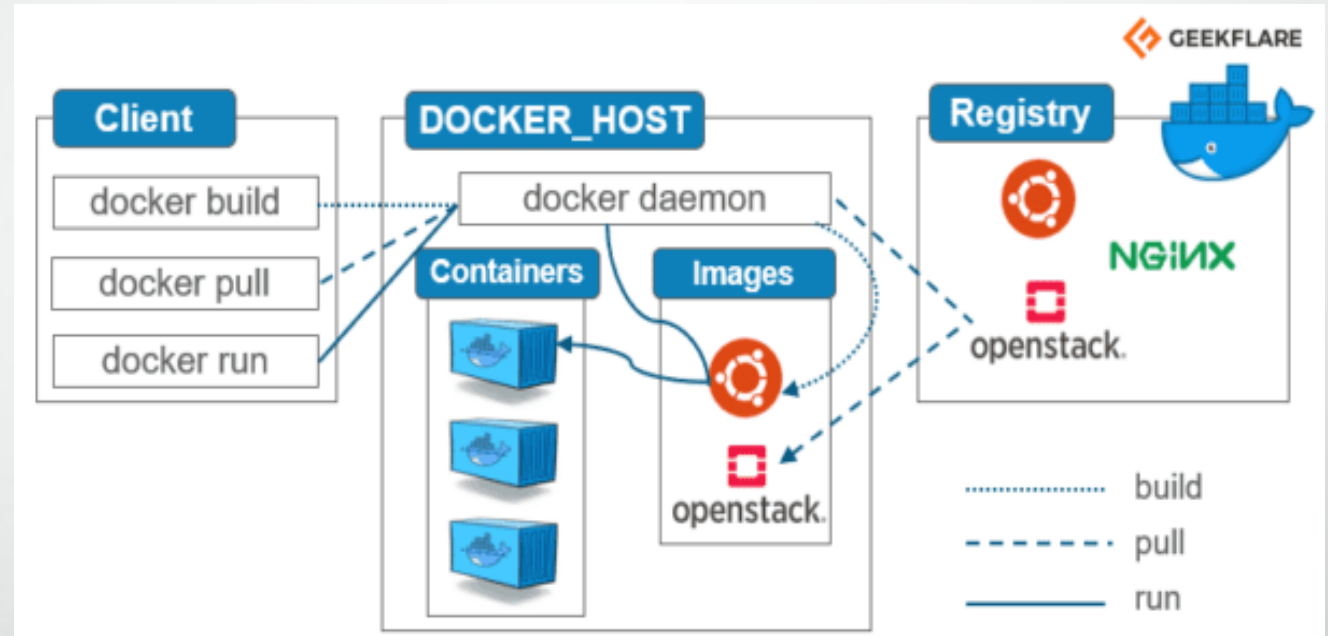


Contenedores



Arquitectura de Docker

- **Cliente Docker (CLI):** envía comandos al daemon.
- **Daemon (dockerd):** construye, ejecuta y gestiona los contenedores.
- **Registry** (Docker Hub, ACR...): almacena y distribuye imágenes.
- **Flujo típico:** docker build → imagen local → docker push al registry → docker pull y docker run en otro entorno.



Imágenes y contenedores

- **Imagen:** plantilla de solo lectura con la aplicación. Se compone de capas (layers).
- **Contenedor:** instancia ejecutable de una imagen. Añade una capa de escritura efímera.
- Ciclo de vida: create → start → run → stop → rm.
- Los cambios en el contenedor no afectan a la imagen.

Dockerfile – Instrucciones principales

FROM: imagen base (ej. FROM node:18-alpine)

WORKDIR: establece el directorio de trabajo

COPY / ADD: copia archivos al contenedor

RUN: ejecuta comandos durante la construcción

EXPOSE: informa del puerto que usa la aplicación

CMD / ENTRYPOINT: comando por defecto al iniciar el contenedor

FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY ..

EXPOSE 3000

CMD ["node", "app.js"]

Construcción de imágenes y caching

```
docker build -t mi-app:v1 .
```

Lógica de Capas (Layers)

- **Ejecución Secuencial:** Docker procesa el Dockerfile línea a línea, creando una capa inmutable por cada instrucción (RUN, COPY, ADD).
- **Mecanismo de Cache:** Si el contenido de una capa y sus predecesoras no han cambiado, Docker **reutiliza** la capa existente (Using cache).
- **Eficiencia Extrema:** Reduce el tiempo de compilación y el ancho de banda al evitar procesar datos redundantes.

Buena Práctica: Ordena las instrucciones de **menor a mayor** frecuencia de cambio.

Comandos básicos

`docker pull nginx`: descarga imagen

`docker run -d -p 8080:80 nginx`: inicia contenedor en segundo plano mapeando puerto 8080

`docker ps`: lista contenedores activos

`docker stop/start/restart <contenedor>`

`docker rm <contenedor>` y `docker rmi <imagen>`

`docker exec -it <contenedor> sh`: accede a terminal dentro del contenedor

`docker logs <contenedor>`: ve los logs



Redes en Docker

bridge (por defecto): red privada interna, comunicación entre contenedores por nombre.

host: el contenedor utiliza directamente la pila de red del anfitrión.

none: sin red.

Mapeo de puertos con `-p` permite exponer servicios al exterior.

`docker network ls`, `docker network create`, etc.

Volúmenes y almacenamiento persistente

Naturaleza Efímera

- Por diseño, los datos dentro de un contenedor son **temporales**. Al eliminar la instancia, los cambios se pierden (Stateless).

Volúmenes (Recomendado)

- Mecanismo gestionado por Docker para persistir datos fuera del ciclo de vida del contenedor. Ideal para **Bases de Datos**
- `docker run -v mi-volumen:/var/lib/mysql mysql`

Bind Mounts

- Útil para bases de datos o configuraciones que deben sobrevivir a recreaciones.
- Es un punto de montaje que vincula una ruta específica del **Host** (tu máquina o servidor) con una ruta dentro del **Contenedor**.
- `docker run -d -v /ruta/en/mi/pc:/app mi-imagen`

MULTI-STAGE BUILDS: EFICIENCIA EN ARTEFACTOS

¿Qué es?

Es una técnica que permite utilizar múltiples instrucciones **FROM** dentro de un mismo Dockerfile. Cada instrucción **FROM** inicia una nueva etapa con una imagen base distinta.

El "Filtro de Artefactos"

La etapa final **solo copia los binarios o archivos estáticos generados** en las etapas previas. Todo lo innecesario (compiladores, código fuente original, cachés de paquetes, herramientas de construcción) se descarta automáticamente.

Dato de Impacto (Comparativa):

- Imagen Tradicional (Node + SDK): ~1.2 GB
- Imagen Multi-Stage (Nginx + Estáticos): ~22 MB
- **Resultado:** Reducción del **98%** en el tamaño de la imagen.

Anatomía de un Dockerfile Multi-Stage

1. Aislamiento de Construcción

- Usamos Node.js completo para compilar React, pero esta capa "pesada" se descarta al final.

2. El Salto Estratégico

- El segundo FROM inicia una imagen limpia de Nginx.

3. Copy --from

- Extracción quirúrgica: solo traemos la carpeta /dist (los archivos estáticos compilados).

```
# Etapa 1: Build (Alias: "build-step")
FROM node:20-alpine AS build-step
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Etapa 2: Run (Servidor ligero)
FROM nginx:stable-alpine
# Copiamos solo el resultado de la etapa anterior
COPY --from=build-step /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Construcción Eficiente de Imágenes

Multi-stage Builds: Separa el entorno de compilación del runtime final para reducir el tamaño drásticamente.

Optimización de Caché: Ordena las instrucciones de menos a más cambiantes para maximizar el reuso de capas.

Uso de .dockerignore: Excluye archivos innecesarios (como logs o node_modules) para acelerar el envío del contexto.

Seguridad y Robustez en Docker

Usuario No-Root: Declara siempre un usuario sin privilegios para mitigar posibles escapes de contenedor.

Etiquetado Explícito: Evita el uso de la etiqueta `:latest` para garantizar que tus builds sean siempre reproducibles.

Gestión de Secretos: Nunca utilices variables de entorno (ENV) para secretos; usa los montajes de BuildKit.

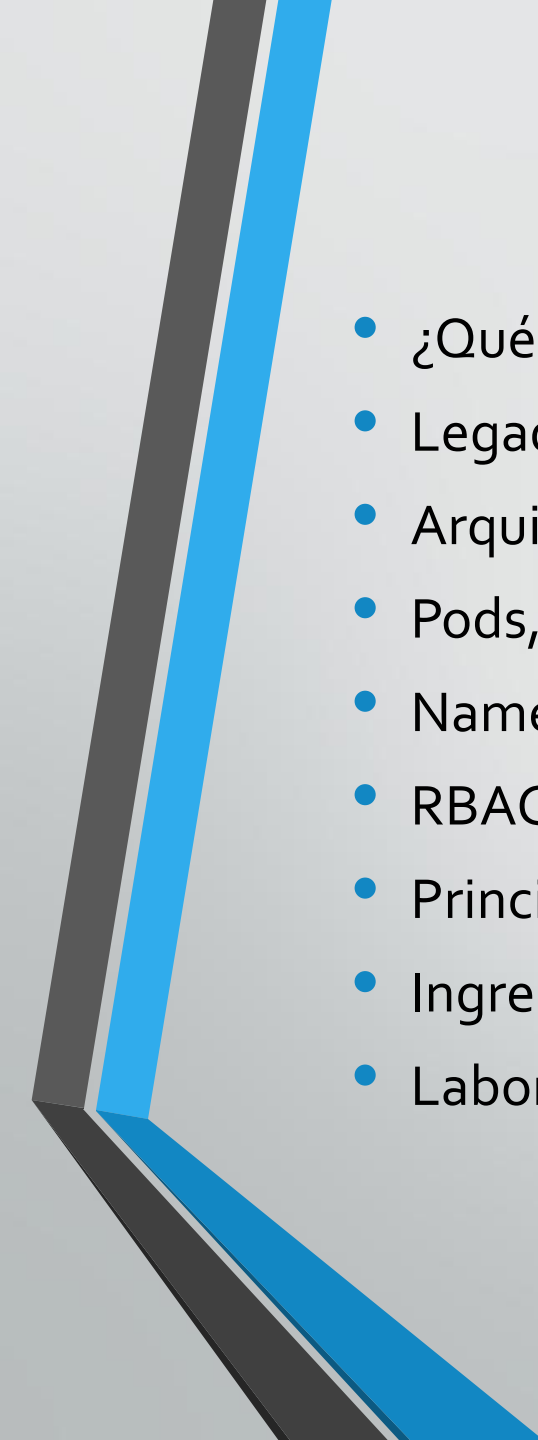
Escaneo Continuo: Integra herramientas como Docker Scout o Trivy en tu CI/CD para detectar CVEs críticas.

Laboratorio Docker

- El laboratorio está en el repositorio: laboratorios/lab1-docker/
- Pasos: construir una imagen simple, ejecutar contenedor, probar mapeo de puertos, explorar comandos básicos.
- Si quieren reproducirlo más tarde, pueden usar cualquier entorno con Docker (labs.iximiuz.com, Play with Docker, o su propio Docker Desktop).
- Ahora vamos a verlo en vivo (yo lo ejecutaré en labs.iximiuz.com).



Fundamentos de Kubernetes

- 
- ¿Qué es Kubernetes? Orquestación de contenedores
 - Legado y gobernanza
 - Arquitectura de Kubernetes (Control Plane y Worker Nodes)
 - Pods, Deployments, Services
 - Namespaces, ConfigMaps y Secrets
 - RBAC básico
 - Principales comandos con kubectl
 - Ingress/Gateway API
 - Laboratorio práctico

¿Qué es Kubernetes?



Automatización Integral

Plataforma open-source diseñada para automatizar el despliegue, escalado dinámico y la gestión operativa de aplicaciones contenerizadas.



Orquestación Inteligente

Gestión avanzada de cargas de trabajo distribuidas en clusters multi-nodo, garantizando alta disponibilidad y optimización de recursos.



Control Declarativo

Enfoque basado en "Estado Deseado": el sistema reconcilia automáticamente la infraestructura real con la definición en YAML.

Legado y Gobernanza

La historia de Kubernetes, cuyo nombre proviene del griego antiguo para "piloto o "timonel", originalmente concebido por Google bajo el nombre en clave "Borg", Kubernetes fue donado a la **Cloud Native Computing Foundation (CNCF)** en 2015.

Hoy es el proyecto graduado más influyente de la fundación, mantenido por una comunidad global de miles de ingenieros y las principales empresas de tecnología del mundo.



Infraestructura y Orquestación

Gestión de Clusters

Kubernetes abstrae la infraestructura subyacente, tratando un conjunto de máquinas físicas o virtuales (Nodos) como un único pool de computación.



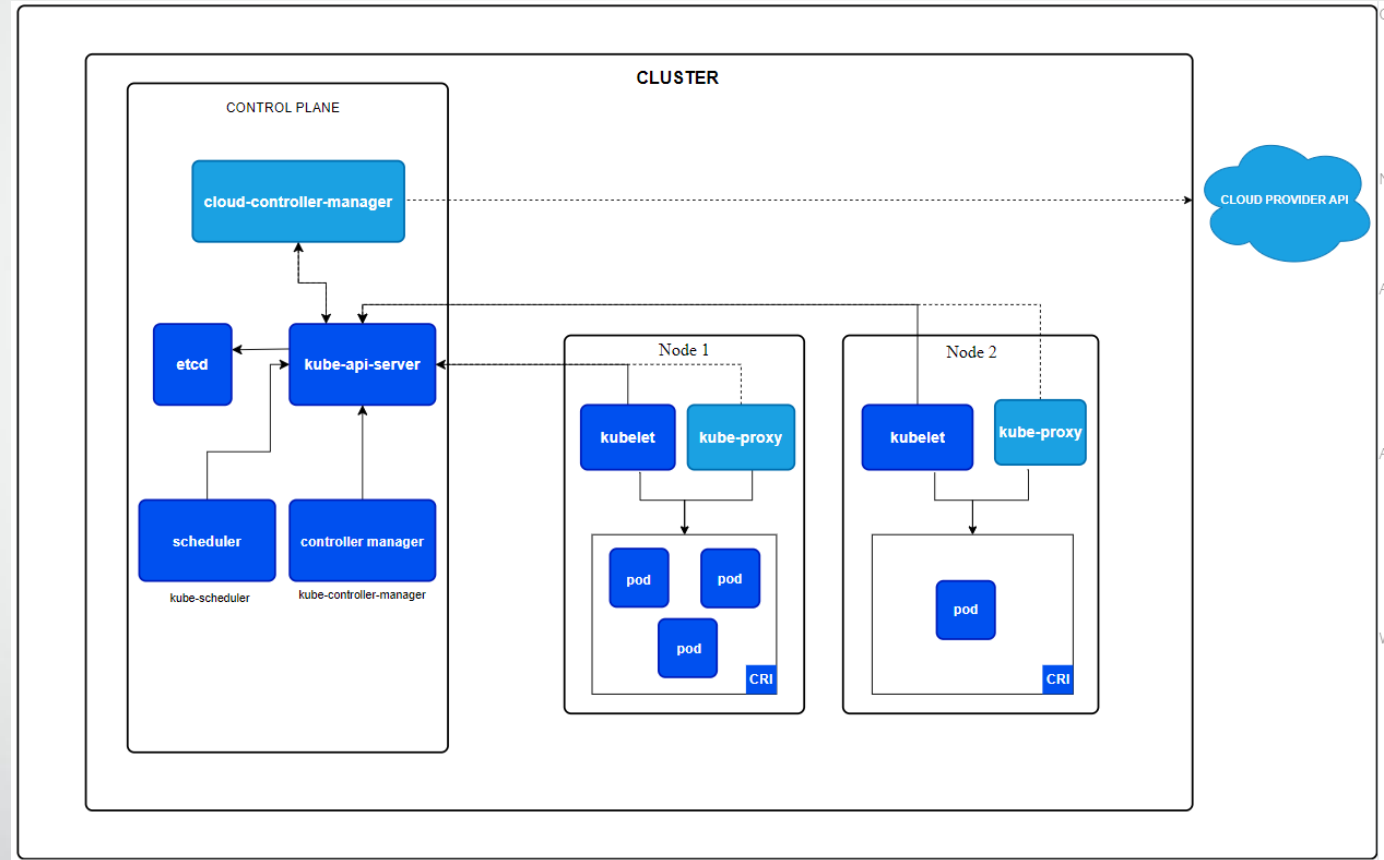
Control Plane: Toma decisiones globales sobre el cluster.



Worker Nodes: Ejecutan las cargas de trabajo (Pods).

Arquitectura de Kubernetes

- **Control Plane:** API Server (punto único de entrada), etcd (almacén de configuración), Scheduler (asigna Pods a nodos), Controller Manager (controladores que reconcilian el estado).
- **Nodos (workers):** Kubelet (agente que ejecuta los contenedores), Kube-proxy (reglas de red), Container Runtime (containerd, CRI-O).
- Comunicación vía API REST, autenticación y autorización.





Namespaces

- Mecanismo para dividir los recursos del clúster entre múltiples equipos o proyectos.
- Proporcionan un ámbito para nombres; los objetos pueden tener el mismo nombre en diferentes namespaces.
- `kubectl get ns`, `kubectl create ns desarrollo`.
- Útil para separar entornos (dev, staging, prod) y aplicar cuotas de recursos.



Pods

Es la unidad mínima desplegable en Kubernetes.

Un Pod puede contener uno o varios contenedores que comparten red y almacenamiento (localhost).

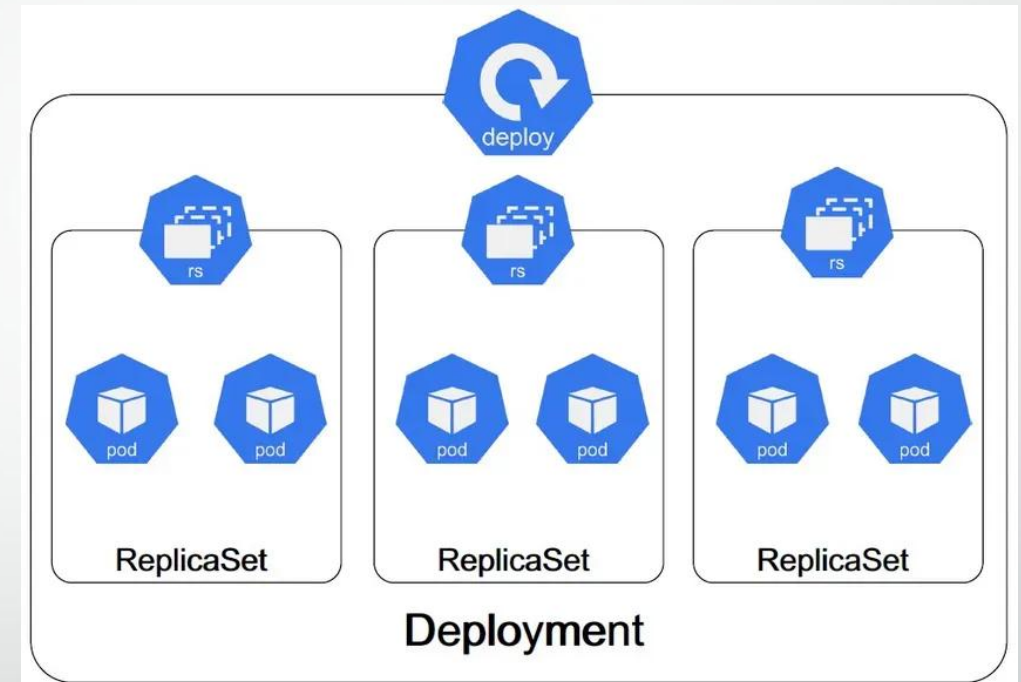
Cada Pod tiene una IP única en el clúster.

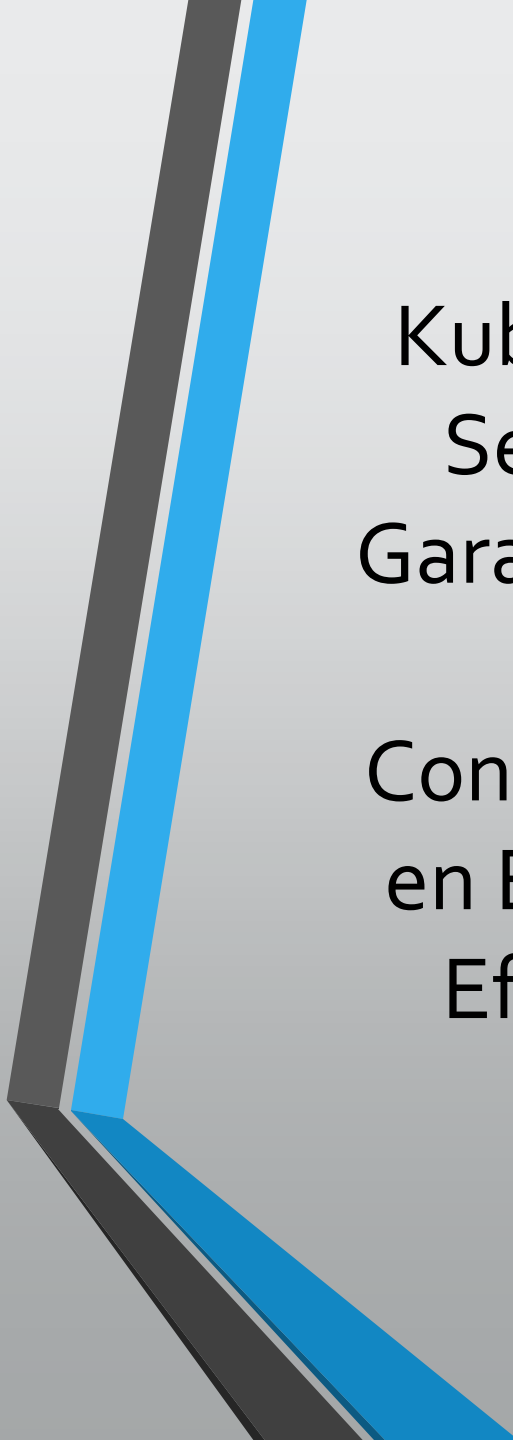
Ciclo de vida:
Pending→Running→Succeeded/Failed.

Los Pods son efímeros; se crean, mueren y se sustituyen.

Deployments

- Controlador que asegura que un número determinado de réplicas de un Pod esté corriendo.
- Permite actualizaciones rodantes (rolling update) y vuelta atrás (rollback).
- Declara el Pod template (qué contenedores, imagen, puertos).
- `kubectl create deployment`, `kubectl scale deployment`, `kubectl rollout undo deployment`





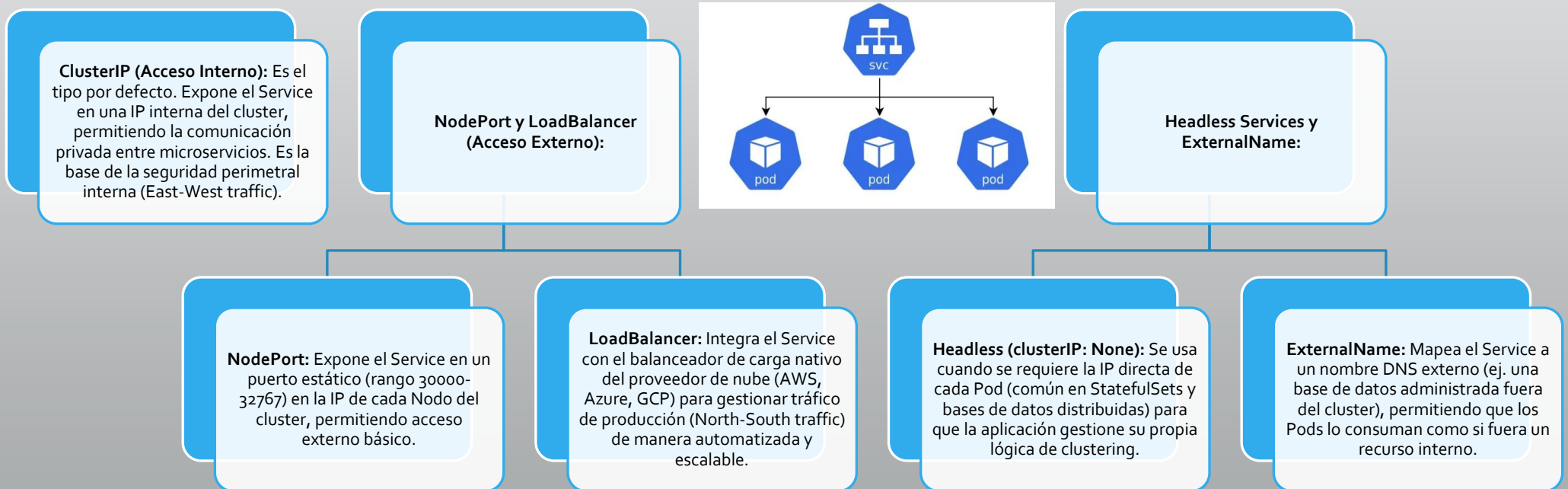
Kubernetes Services: Garantizando la Conectividad en Entornos Efímeros

Abstracción de la Volatilidad de los Pods: Los Pods en Kubernetes son recursos efímeros con direcciones IP dinámicas que cambian tras cada reinicio o despliegue. El **Service** proporciona un punto de acceso estable (IP virtual y nombre DNS) que persiste independientemente del ciclo de vida de los Pods subyacentes.

Mecanismo de Selección (Selectors y Labels): Un Service utiliza *Label Selectors* para identificar y agrupar dinámicamente un conjunto de Pods. Kubernetes gestiona automáticamente un objeto llamado **Endpoints**, que mantiene la lista actualizada de las IPs de los Pods saludables que cumplen con dichos criterios.

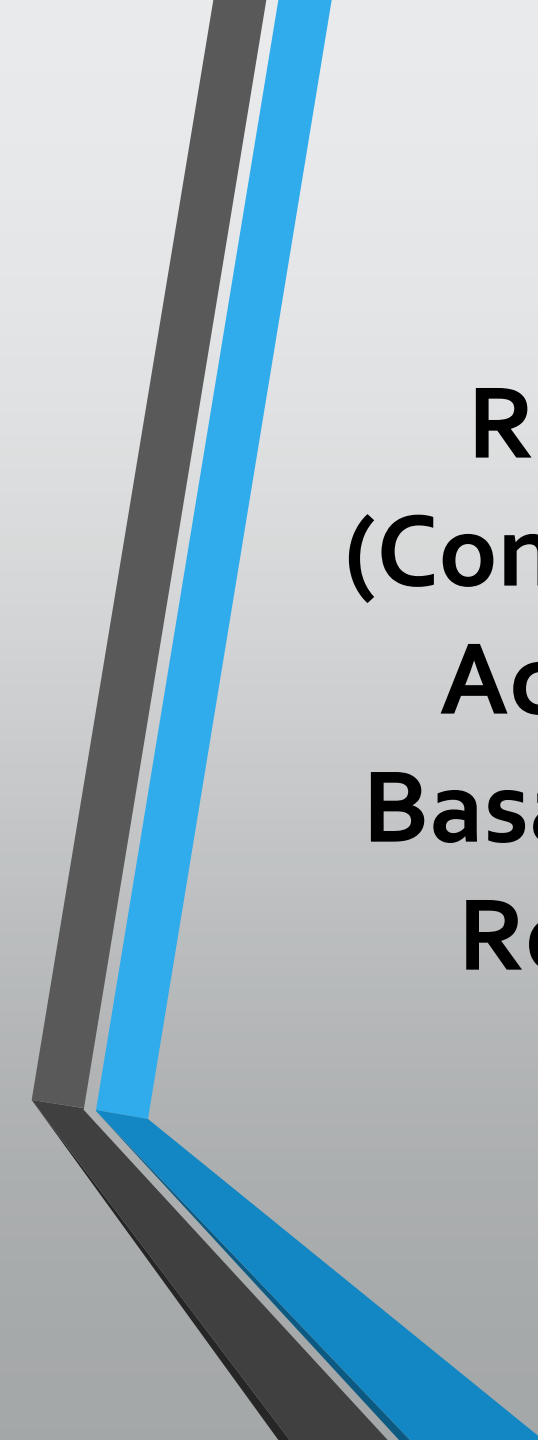
Balaneo de Carga Nativo: El Service actúa como un balanceador de carga básico de Capa 4 (TCP/UDP), distribuyendo de forma equitativa el tráfico entrante entre todos los Pods disponibles que pertenecen al conjunto definido, optimizando el consumo de recursos.

Tipos de Service: Controlando el Alcance del Tráfico



ConfigMaps y Secrets

- **ConfigMap**: almacena configuración no sensible (variables de entorno, archivos de configuración).
- **Secret**: Similar pero para datos sensibles (contraseñas, tokens), codificados en base64 (¡no cifrado por defecto!).
- Se pueden inyectar como variables de entorno (envFrom) o como archivos montados en el Pod.
- Buenas prácticas: no hardcodear configuraciones en las imágenes.



RBAC (Control de Acceso Basado en Roles)

Permite conceder permisos granulares sobre los recursos de la API de Kubernetes.

Role: define permisos dentro de un namespace (p.ej. poder listar Pods).

ClusterRole: a nivel de todo el clúster.

RoleBinding / ClusterRoleBinding: asocia sujetos (usuarios, grupos, ServiceAccounts) a roles.

Verificación: `kubectl auth can-i list pods --as usuario`

Comandos esenciales kubectl

- `kubectl get pods, deployments, services -n <namespace>`
- `kubectl describe pod <nombre> (detalle)`
- `kubectl logs <pod> [-c contenedor]`
- `kubectl exec -it <pod> -- sh`
- `kubectl apply -f <archivo/directorio>`
- `kubectl delete -f <archivo>`
- `kubectl config view, kubectl config use-context`



Ingress vs Gateway API: Conceptos

Ingress (Legacy)

- Diseñado originalmente como un recurso simple para exponer servicios HTTP. Depende fuertemente de anotaciones específicas del proveedor para funciones avanzadas.

Gateway API (Next-Gen)

- Una colección de recursos orientada a roles (GatewayClass, Gateway, HTTPRoute) que ofrece expresividad nativa para tráfico L4 y L7.

Alerta: Retiro de NGINX Controller

MARZO 2026 - RETIRO FINAL

El fin de Ingress-NGINX Community

- El controlador de la comunidad (mantenido por Kubernetes SIG Network) será retirado oficialmente. No recibirá más parches de seguridad ni actualizaciones de compatibilidad con versiones futuras de K8s.

Impacto del NGINX "Deprecation"

- **Riesgo de Seguridad:** El cese de parches deja vulnerabilidades (CVEs) sin resolver.
- **Incompatibilidad:** Nuevas versiones de Kubernetes podrían romper el controlador actual.
- **Mantenimiento Crítico:** El proyecto ha sufrido falta de mantenedores activos durante años.

Ingress vs Gateway API

Característica	Ingress API	Gateway API
Protocolos Soportados	HTTP y HTTPS	HTTP, gRPC, TCP, UDP, TLS
Gestión de Tráfico (Pesos)	Vía anotaciones propietarias	Nativo (HTTPRoute backendRefs)
Modelo de Seguridad	RBAC limitado por recurso	Orientado a roles (Infra vs Apps)
Extensibilidad	Annotations (Caótico)	Filtros y Políticas (Estructurado)

Laboratorio Kubernetes

- Repositorio: laboratorios/lab2-kubernetes/
- Desplegar un Pod, Deployment, Statefulset, Services y probar configuración con ConfigMap y Secret.
- Se puede usar Minikube, kind, killerkoda, kodekloud o labs.iximiuz.com.

Azure Kubernetes Service (AKS)

Orquestación de Contenedores Gestionada en la Nube

¿Qué es AKS y cómo funciona?

Servicio Gestionado: Azure administra el Control Plane (API Server, etcd) sin costo adicional.

Abstracción de Complejidad: Automatiza parches, actualizaciones y mantenimiento de nodos.

Basado en Open Source: Es 100% compatible con la API estándar de Kubernetes.

Ciclo Operativo: El usuario solo se encarga de gestionar los Agent Nodes y el despliegue de apps.

Ventajas de AKS

Escalabilidad Elástica: Soporte nativo para Horizontal Pod Autoscaler (HPA) y Cluster Autoscaler.

Eficiencia de Costos: Solo se paga por las máquinas virtuales (nodos) y almacenamiento consumido.

Seguridad Integrada: Aislamiento de red mediante Azure CNI y políticas de Azure.

Alta Disponibilidad: Soporte para zonas de disponibilidad y SLA del 99.95% para el API Server.

Arquitectura de AKS

Control Plane (Gestionado)

Componentes críticos que Azure opera: kube-apiserver, scheduler, y controller manager.

Nodos de Cliente (Tus VM)

Nodos donde corren tus Pods. Incluyen el kubelet, kube-proxy y el Container Runtime.

Virtual Machine Scale Sets (VMSS): Base de infraestructura para los nodos.

Azure Load Balancer: Creado automáticamente para servicios de tipo LoadBalancer.

Integración con el Ecosistema Azure

Microsoft Entra ID (Azure AD): Control de acceso basado en roles (RBAC) integrado.

Azure Container Registry (ACR): Almacenamiento privado y seguro de imágenes de Docker.

Azure Key Vault: Gestión segura de secretos mediante el driver CSI.

Managed Identities: Los Pods pueden acceder a servicios de Azure sin usar contraseñas.

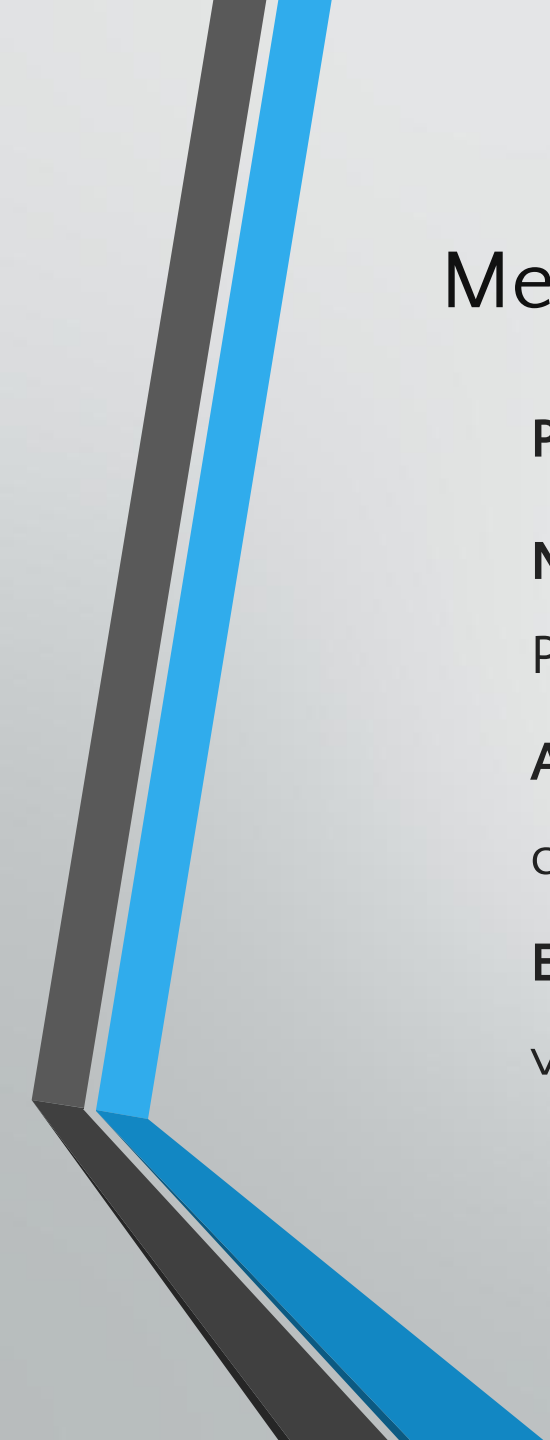
Mejores Prácticas: Operaciones

Infraestructura como Código: Usar Terraform o Bicep para desplegar el clúster.

Uptime SLA: Habilitar el SLA de pago para entornos de producción críticos.

Actualizaciones Programadas: Configurar ventanas de mantenimiento para evitar impacto en horas pico.

Node Pools Múltiples: Separar cargas de trabajo (ej. System vs User) en diferentes pools de nodos.



Mejores Prácticas: Seguridad

Private Clusters: Limitar el acceso al API Server solo a redes internas.

Network Policies: Aplicar principios de "Zero Trust" para restringir tráfico entre Pods.

Azure Policy: Garantizar que todos los despliegues cumplan con estándares de cumplimiento.

Escaneo de Imágenes: Habilitar Defender for Containers para detectar vulnerabilidades en el ACR.

Ejemplo: Creación vía Azure CLI

Crear Resource Group

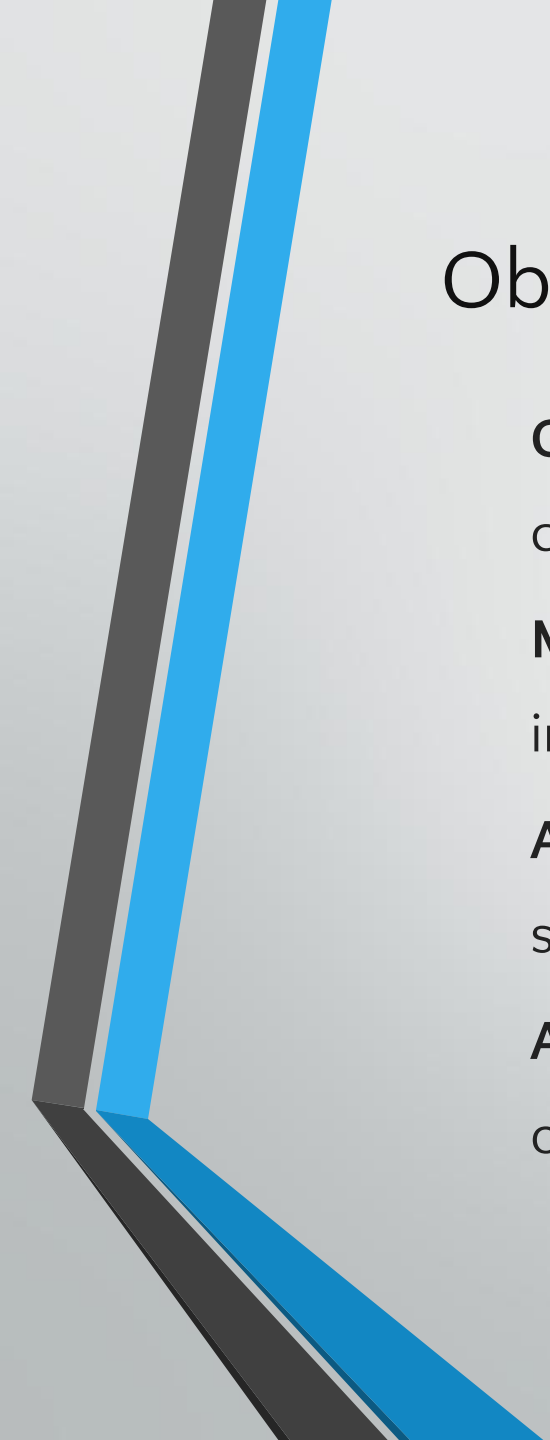
```
az group create \  
  --name MyResourceGroup \  
  --location eastus
```

Crear Clúster AKS

```
az aks create \  
  --resource-group MyResourceGroup \  
  --name MyAKSCluster \  
  --node-count 3 \  
  --enable-addons monitoring \  
  --generate-ssh-keys
```

Obtener Credenciales

```
az aks get-credentials \  
  --resource-group MyResourceGroup \  
  --name MyAKSCluster
```



Observabilidad y Monitoreo

Container Insights: Visualización detallada de CPU, memoria y salud de los contenedores.

Managed Prometheus/Grafana: Recolección nativa de métricas sin gestionar infraestructura.

Azure Log Analytics: Repositorio centralizado para logs de aplicación y del sistema.

Alertas Inteligentes: Configurar notificaciones basadas en el estado del clúster o latencia de apps.

¡Muchas Gracias!

Sesión de preguntas y respuestas



linkedin.com/in/edwin-Bermudez-cuervo



ebermudez@coem.co



github.com/edwinbermudez91